

```
traces_sample_rate=0.1,
)
```

Scaling databases horizontally

As data grows, a single database server becomes insufficient. Horizontal scaling distributes data across multiple servers through sharding or read replicas.

Database sharding divides data by a sharding key—user ID, geographic region, or product category. Django's database routers direct queries to the appropriate shard based on this key. Each shard operates independently, allowing linear scaling.

```
#settings.py
DATABASES = {
    'default': {'ENGINE': 'django.db.backends.postgresql',
    ...},
    'shard_1': {'ENGINE': 'django.db.backends.postgresql',
    ...},
    'shard_2': {'ENGINE': 'django.db.backends.postgresql',
    ...},
    }
#Database router
class ProductShardRouter:
    def db_for_read(self, model, **hints):
        if model._meta.app_label == 'products':
            product_id = hints.get('instance').id
            return f'shard_{(product_id % 2) + 1}'
        return 'default'
DATABASE_ROUTERS = ['myapp.routers.ProductShardRouter']
```

Read replicas handle read operations, leaving the primary database for writes. Configure multiple replica databases, then use a database router to send read queries to replicas randomly or round-robin. This distributes read load while maintaining a single source of truth for writes.

```
#settings.py
DATABASES = {
    'default': {'HOST': 'primary.db.server', ...},
    'replica_1': {'HOST': 'replica1.db.server', ...},
    'replica_2': {'HOST': 'replica2.db.server', ...},
    }
#Database router
class ReadReplicaRouter:
    def db_for_read(self, model, **hints):
        import random
        return random.choice(['replica_1', 'replica_2'])
    def db_for_write(self, model, **hints):
        return 'default'
```

Scaling databases vertically

Vertical scaling upgrades hardware resources—CPU, RAM, storage—on existing servers. Before implementing complex horizontal scaling, maximise single-server performance through configuration tuning and hardware upgrades.

PostgreSQL configuration significantly impacts performance. Tune `shared_buffers`, `effective_cache_size`, `work_mem`, and connection limits based on available RAM. These settings determine how PostgreSQL uses memory for caching and query execution.

```
#!/etc/postgresql/14/main/postgresql.conf
shared_buffers = 4GB
effective_cache_size = 12GB
work_mem = 50MB
max_connections = 200

#Restart PostgreSQL
sudo systemctl restart postgresql
```

Monitor query performance using PostgreSQL's `pg_stat_statements` extension. It tracks query execution times, call counts, and resource usage. Identify slow queries and optimise them through better indexing, query rewriting, or schema changes.

Upgrading to faster storage like NVMe SSDs dramatically improves I/O-bound workloads. Database performance often depends on disk speed, making storage upgrades highly effective.

Real-world case study

A growing e-commerce platform faced performance challenges during flash sales. With 50,000 concurrent users, response times exceeded 10 seconds, and servers crashed regularly.

Initial problems included N+1 queries on product listings, no caching layer, synchronous payment processing blocking workers, large JSON payloads with unnecessary data, and a single database server at 95% CPU utilisation.

The team implemented fixes in five phases. Database optimisation added `select_related` and `prefetch_related` to eliminate N+1 queries, created indexes on frequently filtered columns, and implemented PgBouncer for connection pooling.

Caching layer deployment added Redis with 8GB memory, cached product listings with 15-minute expiration, cached category hierarchies for one hour, and implemented signal-based cache invalidation.

Asynchronous processing offloaded payment processing to Celery workers, moved email notifications to background tasks, and added real-time order status updates with Django Channels.

■■ *Continued to page....79*